# "Code was never the hard part" is an insult to all programmers

*Senko Rašić*

The software development profession is in the midst of upheaval. Nobody knows how the AI revolution will play out in the end, but it is clear many aspects of work and life will be transformed —including programming.

One of the comments I hear often lately boils down to "*LLMs may be good at coding, but software was never the hard part*" and "*coding is easy, it's figuring out what to code that's hard*".

I believe that's a gross insult to all programmers everywhere.

## If coding is easy...

If coding is easy, how come programmers were in high demand, and have demanded large salaries for years (even before ZIRP)? Why was there so much stress, overwork and burnout even before AI started churning out 5000-line PRs? Why did companies seek 10x ninja rockstar coders and subject them to leetcode interviews—surely, a junior fresh out of college could churn out something if it's so easy?

If coding is easy, why do we have doorstoppers like Clean Code and The Pragmatic Programmer? Is The Art of Computer Programming a light summer read? Is SICP a coffee-table book? Why do we have bootcamps or even whole college degrees dedicated to it?

If coding is easy, was Carmack just at the right place at the right time? Why do we consider Fabrice Bellard a genius?

If coding is easy, why are people angry at AI (or anyone else) copying their code? Why do they act like they've poured their sweat, soul, and copious amounts of time into something so trivial?

If coding is easy, why do many now feel like their identity and professional purpose are being stripped away from them?

If coding is easy, why is software so damn buggy?

## If figuring out what to build is the hard part...

If deciding what to build is the hard part, why do so many product managers seem clueless? Why aren't there rigorous 10-step interviews for them? Why aren't they getting paid more than the developers?

If deciding what to build is the hard part, why aren't market researchers, usability experts and—hell, customer success—considered rockstars in a software company? If "understanding the

customer” is harder, why are business analysts looked down on as pencil pushers?

If implementation is easy and finding demand is harder, why are programmers upset when the salespeople promise a new feature to a customer to close the sale? They've found a genuine demand, something people will pay for!

If coding is easy, why doesn't everyone just build ten variations of a thing and see which pans out?

Another cliché comment is “*most work in software development is talking to stakeholders, understanding the customer's needs, and having clarity on the priorities*”.

I have met many programmers throughout my career, and very few of them want to talk to stakeholders, much less customers (exceptions are freelancers and founders, especially of software development shops). And, “having clarity on the priorities” boils down to “just tell me what to do and don't switch it up every two days”.

Some software developers do say “I don't write code, I solve customer's problems”. But then they turn around and start to opine on monads, memory safety, and DRY principles, while their understanding of the customer is a made-up “user persona”, and they think “affordance” is the money your parents used to give you on weekends so you could go out and have a good time.

Yet others will say “Software development is theory building”. Programs are actually proofs (as in, mathematical proofs). Every commit should tell a story. And solving a customer's problem by FTPing a PHP file is a cardinal sin.

I don't mean to imply there are no developers that simultaneously care deeply about the craft of software development and really empathize with the customer. I do believe they might want to see a professional about a split personality disorder, tho. (*post-scriptum* edit: this sentence was way overboard; I meant it as tongue-in-cheek “very few people can do that” and I do actually advocate for that in the next section. *mea culpa*)

# What *is* important?

I do believe that talking to users, understanding their experience, empathizing with them, solving customers' problems and having all the stakeholders on the same page is critical to the success of a software project.

I also believe that creating good code is a craft that requires skill, patience, attention to detail, experience and wisdom, and that it will continue to be relevant in the times ahead.

¿Por qué no los dos?

To the extent that we can pull it off, I think we should aim for both. A deep understanding of the system we're building, together with a deep understanding of *why* we're building it.

Loudly proclaiming that “code is easy” or, at the opposite end, “code is art, a creative human expression that cannot be automated”, is just burying our heads in the sand.

It's cope. And you don't want cope, you want to thrive.

By this, I don't mean “jump on the LLM bandwagon.” I don't mean “become a manager of fleets of

AI agents." I also don't mean "AI-generated code is stolen slop garbage, fight it with tooth and nail, the bubble will pop soon enough anyways."

But do recognize we're in the middle of an industry-wide tectonic change. We need to figure out how to adapt. We need to understand what is likely to change and what never changes.

## What doesn't change?

Software will be getting more complex. Software will always need maintenance: bit-rot is a fact of life. So is entropy. Technology (hardware and software) will move forward, for better or worse. The tower (skyscraper?) of abstractions grows ever higher.

Users will always want more and be prepared to spend less. They still won't know how to relay their needs and wants. Worse, they still won't know exactly what they want. The disconnect between the customers (who actually pay for the software) and users (who use it) will still be here, as will the tension between the needs of the business and the needs of its customers.

Also: there will never be a shortage of snake oil salesmen. Tech du jour comes and goes (I'm still waiting for the new VR renaissance!)

## What changes?

Programmers have been in the business of disrupting our own industry since the beginning. Nobody uses punch-cards any more. Very few people need to code in assembly, or COBOL. Those decades spent fighting memory bugs in C or C++, with the scars to prove it, are worthless in the age of Rust, Go, Python and JavaScript.

I'm old enough to appreciate valgrind or remember `mysql_real_escape_string()` from the PHP4 era—stuff I'll never again need in my life. And that wasn't even so long ago! I narrowly missed the dBase, Clipper, HyperCard and Access era, technologies which I can still spot operating in shops, cafes, or a dusty, once beige and now golden-brown, midi-tower still happily running some bespoke biz solution (backups? what backups?)

## How do we thrive?

Accept that change happens. Be equal parts curious and critical about the new stuff.

Understand there's a lot of hype and try to discriminate between hot air and what really works (and to what extent). Also be aware of ever-shifting goalposts: stand back and look at the past year, or five, and assess the velocity of change (technical, economic, societal).

Your role and your responsibilities *will* be changing. Be willing to invest time and energy into better understanding fields or roles adjacent to yours.

If you're a senior developer, don't just find solace in deepening your expertise. Learn about user experience, customer interviews, or business strategies for the companies in your domain. It will help you gain a better appreciation of all the work done to put a piece of software into users' hands, whether or not you'll actually ever have to do any of those other bits.

If you're just starting or are junior in your role: invest in deepening your understanding of how software works. Understanding pointers, recursion, or memory hierarchy will help you even if you're a JavaScript developer. Understanding network protocols and how HTTP works will be useful even if you're building WordPress plugins. Do leetcode and learn about algorithms and data structures even if you don't need to. Don't be afraid to ask *why* and *how exactly*.

For inspiration, here are a few books and other resources that might be helpful:

- Structure and Interpretation of Computer Programs (PDF)
- Cracking the Coding Interview
- The Mythical Man-Month
- Working Backwards
- Team Topologies
- 7 Powers
- The Soul of a New Machine
- Obviously Awesome
- The Design of Everyday Things
- Don't Make Me Think
- Continuous Discovery Habits
- The Mom Test

# One more thing

Whoever you are, don't outsource your understanding, judgement, empathy and taste to AI. Don't abdicate your responsibility. Don't be a meat proxy.

---

PS. Lots of interesting and insightful comments over at Hacker News and Lobsters—the post really hit a nerve. Fascinating how many different experiences people have and the varying definitions of *coding*, *programming*, *development* and *engineering* they use.